

```

      ||      @@_2@@      ||
    @_1@    @@@_3@@@
  @@@@_6@@@@ @@@_4@@@@ @@@@_5@@@@
--snip--
      A          B          C
      ||      ||      ||
      ||      @_1@      ||
      ||      @@_2@@      ||
      ||      @@@_3@@@      ||
      ||      @@@_4@@@@      ||
      ||      @@@@_5@@@@      ||
      ||      @@@@@_6@@@@      ||
      A          B          C

```

The Python version has an interactive mode too, where you can solve the puzzle yourself. Uncomment the lines of code at the end of *towerOfHanoiSolver.py* to play the interactive version.

You can start by running the program with the smaller cases by setting the `TOTAL_DISKS` constant ❶ at the top of the program to 1 or 2. In our program, a list of integers in Python and an array of integers in JavaScript represent a pole. The integer represents a disk, with larger integers representing larger disks. The integer at the start of the list or array is at the bottom of the pole, and the integer at the end is at the pole's top. For example, [6, 5, 4, 3, 2, 1] represents the starting pole with six disks with the largest on the bottom, while [] represents a pole with no disks. The `TOWERS` variable contains three of these lists ❷.

The base case merely moves the smallest disk from the start pole to the end pole ❸. The recursive case for a tower of n disks carries out three steps: solving the $n - 1$ case ❹, moving the n th disk ❺, and then solving the $n - 1$ case again ❻.